

SMART LIGHTING CONTROL SYSTEM

(for 230 V AC)

TECHNICAL REPORT

Team Members

Amin Yakoubi and R. Geraldo Chikoti

Course

Practice Enterprise

Table of Contents

Executive Summary	3
1. System Architecture and Operating Principle	3
2. Master Communication and User Interface	4
3. OLED Status Display	12
4. Slave Firmware and Phase-Control Timing	14
5. Zero-Cross Detection	16
6. TRIAC Power Stage	18
7. Power-Supply and Protection Circuits	21
8. PCB Implementation	25
9. Mechanical and Enclosure Design	29
10. Development Problems and Troubleshooting	31
11. Integrated System Behaviour	33
12. Verification and Recommended Tests	34
13. Conclusion	34
14. References	35

Executive Summary

The Smart Lighting Control System is a distributed embedded platform for controlling 230 V AC lighting loads. A central STM32 master receives commands from a mobile application through an HC-05 Bluetooth interface, provides local control through push buttons, displays system status on an SSD1306 OLED, and communicates with dimmer slaves over I²C. Each slave detects the AC zero crossing and triggers two independent TRIAC channels after a programmable delay.

Safety notice Parts of the system are connected directly to 230 V AC. The calculations and design explanations in this report are not a certification. Fusing, earthing, enclosure flame rating, creepage, clearance, component ratings, thermal behaviour, wiring and test methods must be validated against the applicable standards before operation.

Subsystem	Function	Key implementation
Bluetooth / UART	Receives app commands	USART, byte-by-byte interrupt reception
Master I ² C	Controls and polls slave nodes	Two-byte commands; dynamically assigned slave addresses
OLED	Shows slave and dimmer status	SSD1306, 128 × 64 pixels
Zero-cross detector	Provides a phase reference	H11AA1 on 230 V, 50 Hz
Power stage	Controls AC power	Random-phase optotriac and BTA16 TRIAC
Power conversion	Creates logic rails	12 V → 5 V buck; 5 V → 3.3 V LDO
Mechanical system	Separates and protects electronics	Custom master and slave enclosures

1. System Architecture and Operating Principle

The system separates user interaction, low-voltage control and mains switching. The master node interprets user commands and distributes dim values. Slave nodes convert those values into timed TRIAC gate pulses. This modular approach allows additional slave units to be added without duplicating the complete user interface.

Stage	Data or action
1	The mobile application sends a command such as 1:0:128.
2	USART receives the message one byte at a time and marks it as complete.
3	The master parses the slave, channel and PWM value.
4	The PWM value is converted to a dim value and sent over I ² C.

Stage	Data or action
5	The slave stores the value for TRIAC channel 1 or 2.
6	A zero-cross interrupt resets the timing reference every half-cycle.
7	TIM2 waits for the calculated delay and generates a short gate pulse.
8	The TRIAC conducts until load current falls below its holding current.

2. Master Communication and User Interface

2.1 Bluetooth and UART message reception

The UART receive callback handles one byte per interrupt. Ordinary characters are appended to rxBuffer. A carriage return or line feed terminates the message, adds the C string null character, copies the result to btMessage, sets btNewData, resets the buffer index and immediately restarts interrupt reception.

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1) {
        if (rxByte == '\n' || rxByte == '\r') {
            if (rxIndex > 0) {
                rxBuffer[rxIndex] = '\0';
                strncpy(btMessage, rxBuffer, sizeof(btMessage) - 1);
                btMessage[sizeof(btMessage) - 1] = '\0';
                btNewData = 1;
                rxIndex = 0;
            }
        } else if (rxIndex < sizeof(rxBuffer) - 1) {
            rxBuffer[rxIndex++] = rxByte;
        } else {
            rxIndex = 0;
        }
        HAL_UART_Receive_IT(&huart1, &rxByte, 1);
    }
}
```

Variable	Purpose
rxByte	The byte currently received.
rxBuffer	Temporary buffer for the message being assembled.
rxIndex	Position of the next byte in rxBuffer.
btMessage	Complete message ready for parsing.
btNewData	Flag indicating that a complete message is available.

2.2 I²C dim command

Send_Dim_To_Slave() converts a PWM value from 0–255 into an inverted dim value and transmits a command-value pair. Command 0x02 selects TRIAC channel 1 and command 0x03 selects TRIAC channel 2. The value is limited to the operating range 5–95 in this master-side example.

```
void Send_Dim_To_Slave(uint8_t channel, uint8_t pwm)
{
    uint8_t addr = 0x12;
    uint8_t dim = 100 - ((uint32_t)pwm * 100 / 255);
    if (dim < 5) dim = 5;
    if (dim > 95) dim = 95;
    uint8_t cmd = (channel == 0) ? 0x02 : 0x03;
    uint8_t data[2] = {cmd, dim};
    HAL_I2C_Master_Transmit(&hi2c2, addr << 1, data, 2, 100);
}
```

PWM input	Raw dim value	Transmitted dim value
0	100	95
64	75	75
128	≈ 50	≈ 50
192	≈ 25	≈ 25
255	0	5

2.3 Dynamic slave addressing and local controls

Identical slave boards start in an address-assignment state. The master activates an ENABLE line and transmits a unique hardware ID together with a new I²C address. A matching slave reinitializes its peripheral with that address and then enters normal operation. Separate addresses allow each slave to be polled and controlled independently.

- I²C1 on PB6/PB7: SSD1306 OLED display.
- I²C2 on PB10/PB11: dimmer slave bus.
- PB2: shared ENABLE line used during address assignment.
- PB5 and PB4: active-low local push buttons with internal pull-ups.
- USART signals: HC-05 Bluetooth data link; STATE input indicates connection status.
- PA13/PA14: SWDIO and SWCLK for programming and debugging.

2.4 HC-05 command parser

This is one of the most important master functions because it connects the mobile application to the dimmer network. It accepts the text format slave:channel:value, validates all three fields, checks whether the requested slave is available and then calls the existing Send_Dim_To_Slave() function. The UART receive callback and Send_Dim_To_Slave() are already shown earlier and are therefore not repeated here.

```
void HC05_ProcessData(char *data)
{
    int slave, channel, value;
```

```
if (!hc05_connected)
    return;

if (sscanf(data, "%d:%d:%d", &slave, &channel, &value) != 3)
    return;

if (slave < 1 || slave > 2) return;
if (channel < 0 || channel > 1) return;
if (value < 1) value = 1;
if (value > 255) value = 255;

if (slave == 1 && !slave1_found) return;
if (slave == 2 && !slave2_found) return;

Send_Dim_To_Slave((uint8_t)slave,
                  (uint8_t)channel,
                  (uint8_t)value);
}
```

Why this code matters Without validation, an invalid slave number, channel or value could be transmitted on I²C. This function acts as the controlled entry point for all dim commands from the app.

2.5 Bluetooth connection-state control

The HC-05 STATE output is read on PB15. When the app connects, local button control is disabled and the local button values are reset to zero. Zero is sent once to each available slave so the app starts from a known condition. The one-shot flags are essential: repeatedly sending zero would overwrite every new dim value coming from the app.

```
void Update_HC05_State(void)
{
    uint8_t new_state =
        (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_15) == GPIO_PIN_SET);

    if (new_state && !hc05_prev_connected) {
        dim_slave1 = dim_slave2 = 0;
        direction_slave1 = direction_slave2 = 1;
        hc05_zero_sent_s1 = hc05_zero_sent_s2 = 0;
        OLED_Show_Dim(1, 0);
        OLED_Show_Dim(2, 0);
    }

    if (new_state) {
        if (slave1_found && !hc05_zero_sent_s1)
            if (Send_Button_Dim(0x12, 0) == HAL_OK)
                hc05_zero_sent_s1 = 1;

        if (slave2_found && !hc05_zero_sent_s2)
            if (Send_Button_Dim(0x13, 0) == HAL_OK)
                hc05_zero_sent_s2 = 1;
    }

    if (!new_state && hc05_prev_connected) {
        old_switch1 = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_5);
        old_switch2 = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4);
    }
}
```

```

    hc05_connected = new_state;
    hc05_prev_connected = new_state;
}

```

Why this code matters It prevents the app and physical buttons from controlling the same slave at the same time. Reading the button states again after disconnection also prevents a false button press.

2.6 Local button dimming

Process_Buttons() detects a press on the transition from released HIGH to pressed LOW. PB5 controls slave 2 and PB4 controls slave 1. Each successful press changes the local dim value by five. At 95 the direction reverses, and at 0 it changes upward again. The complete source performs the same sequence separately for both buttons; the fragment below shows the essential slave-2 branch without repeating the nearly identical slave-1 branch.

```

void Process_Buttons(void)
{
    GPIO_PinState sw1 = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_5);
    GPIO_PinState sw2 = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4);

    if (hc05_connected) {
        dim_slave1 = dim_slave2 = 0;
        direction_slave1 = direction_slave2 = 1;
        old_switch1 = sw1;
        old_switch2 = sw2;
        return;
    }

    if (old_switch1 == GPIO_PIN_SET && sw1 == GPIO_PIN_RESET &&
        slave2_found) {
        if (direction_slave2) {
            if (dim_slave2 < 95) dim_slave2 += 5;
            else { direction_slave2 = 0; dim_slave2 -= 5; }
        } else {
            if (dim_slave2 > 0) dim_slave2 -= 5;
            else { direction_slave2 = 1; dim_slave2 += 5; }
        }

        if (Send_Button_Dim(0x13, dim_slave2) == HAL_OK)
            OLED_Show_Dim(2, dim_slave2);
        else
            slave2_found = 0;

        HAL_Delay(50);          // basic debounce
    }

    /* PB4 uses the same logic for slave 1 at address 0x12. */
    old_switch1 = sw1;
    old_switch2 = sw2;
}

```

Why this code matters The edge check allows one change per press, the direction flag creates an up/down dim sequence, and the slave_found check prevents communication attempts to a missing board.

2.7 I²C bus recovery

If a slave resets during a transfer, SDA can remain low and the I²C peripheral can stay busy. I2C_ClearBus() temporarily changes PB10 and PB11 to open-drain GPIO outputs, generates nine clock pulses and then creates a STOP condition. Finally, I²C2 is initialized again.

```
void I2C_ClearBus(void)
{
    GPIO_InitTypeDef gpio = {0};
    HAL_I2C_DeInit(&hi2c2);

    gpio.Pin = GPIO_PIN_10 | GPIO_PIN_11;
    gpio.Mode = GPIO_MODE_OUTPUT_OD;
    gpio.Pull = GPIO_PULLUP;
    gpio.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOB, &gpio);

    for (int i = 0; i < 9; i++) {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_SET);
        HAL_Delay(1);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_RESET);
        HAL_Delay(1);
    }

    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_SET);
    MX_I2C2_Init();
}
```

Why nine pulses? A slave can be waiting to complete a byte transfer. Up to eight data bits and one acknowledge bit may need to be clocked before the slave releases SDA.

2.8 Assigning unique slave addresses

Both slave boards can start in the same temporary address state. The master raises ENABLE and sends the hardware ID together with the required new address. Only the slave whose MY_ID matches accepts the address. The bus is recovered first if the HAL peripheral reports BUSY or another I²C error.

```
void Assign_Address(uint8_t id, uint8_t newAddr)
{
    uint8_t data[2] = {id, newAddr};

    if (hi2c2.State == HAL_I2C_STATE_BUSY ||
        HAL_I2C_GetError(&hi2c2) != HAL_I2C_ERROR_NONE)
        I2C_ClearBus();

    HAL_GPIO_WritePin(enable_GPIO_Port, enable_Pin, GPIO_PIN_SET);
    HAL_Delay(50);

    if (HAL_I2C_Master_Transmit(&hi2c2, 0x00 << 1,
                                data, 2, 200) != HAL_OK)
        I2C_ClearBus();

    HAL_Delay(50);
    HAL_GPIO_WritePin(enable_GPIO_Port, enable_Pin, GPIO_PIN_RESET);
}
```



```
    HAL_Delay(100);
}
```

Hardware ID	Assigned I ² C address	Node
0xA2	0x12	Slave 1
0xA1	0x13	Slave 2

2.9 Main control loop and slave polling

The main loop coordinates the complete master project. Fast tasks run every iteration: read HC-05 STATE, process buttons and handle a completed Bluetooth message. Timed tasks use HAL_GetTick(): missing addresses are assigned every two seconds and both slaves are polled every 300 ms. Polling updates slave1_found and slave2_found, refreshes the OLED and returns connection status to the app.

```
while (1)
{
    Update_HC05_State();
    Process_Buttons();

    if (btNewData) {
        btNewData = 0;
        HC05_ProcessData(btMessage);
    }

    if (HAL_GetTick() - last_scan >= 2000) {
        last_scan = HAL_GetTick();
        if (!slave1_found) Assign_Address(0xA2, 0x12);
        if (!slave2_found) Assign_Address(0xA1, 0x13);
    }

    if (HAL_GetTick() - last_poll >= 300) {
        last_poll = HAL_GetTick();
        Poll_Slave(0x12, &slave1_found); // status + OLED update
        Poll_Slave(0x13, &slave2_found);
        Send_Status_To_App(slave1_found, slave2_found);
    }
}
```

Important note Poll_Slave() and Send_Status_To_App() are descriptive helper names used here to keep the report readable. In the supplied source, their operations are written directly inside main().

2.10 HC-05 Bluetooth module

The HC-05 is a Bluetooth Classic serial module that acts as a wireless replacement for a UART cable. The mobile phone pairs with the module and sends characters over a serial Bluetooth profile. On the hardware side, the module presents the received characters on its TX pin and accepts outgoing characters on its RX pin. The STM32 therefore communicates with the HC-05 through an ordinary asynchronous UART peripheral rather than through a special Bluetooth protocol stack.

HC-05 connection	Function in the project
------------------	-------------------------

HC-05 connection	Function in the project
VCC / GND	Powers the module and establishes the electrical reference.
TX → STM32 RX	Carries app commands from the HC-05 to the master MCU.
RX ← STM32 TX	Carries status and debug data from the master to the HC-05.
STATE → PB15	Indicates whether a Bluetooth connection is active.
Serial data format	Text messages such as 1:0:128 terminated by a line ending.

The module is used only as a communication bridge. It does not calculate dim levels and does not control the TRIACs directly. The STM32 validates the received text, converts the app value into the project dim range and sends the final command to the selected slave over I²C.

Logic-level check The exact HC-05 breakout board must be checked before connection. Many boards accept a 5 V supply but still use approximately 3.3 V UART logic. The STM32 and HC-05 must share a stable low-voltage ground reference during UART testing.

2.11 Mobile application with Flutter and Dart

The mobile application was developed with Flutter and the Dart programming language. Flutter is a cross-platform user-interface framework: one project can provide the screens, buttons and sliders for an Android application. Dart contains the program logic that scans for the HC-05, opens the serial Bluetooth connection, formats dim commands and updates the visible connection and slave status.

Application part	Role
Connection screen	Requests permissions, searches for or selects the paired HC-05 and opens the Bluetooth link.
PWM control screen	Lets the user select a slave, dimmer channel and value.
Command formatter	Builds slave:channel:value followed by a line ending.
Bluetooth listener	Receives master status messages and updates the interface.
State management	Keeps connection, slave status and slider values synchronized with the UI.

```
String command = '$slave:$channel:$value\n';
connection.output.add(Uint8List.fromList(utf8.encode(command)));
await connection.output.allSent;
```

For example, 1:0:128 selects slave 1, dimmer channel 1 and app value 128. The newline is important because the STM32 UART callback uses a line ending to decide that the complete message is ready. The app separates the user interface from the embedded timing: Flutter chooses the requested value, while the slave MCU still performs the precise zero-cross and TRIAC timing.

3. OLED Status Display

The master uses a 128 × 64 SSD1306 OLED on I²C1. The display driver separates configuration commands from image data. A control byte of 0x00 identifies a controller command, while 0x40 identifies display-memory data.

3.1 Driver functions

Function	Operation
OLED_WriteCommand()	Transmits 0x00 followed by one SSD1306 command byte.
OLED_WriteData()	Transmits 0x40 followed by up to 128 bytes of display data.
OLED_Print()	Maps supported characters to five-column 5 × 7 glyphs and adds one blank spacing column.
OLED_Init()	Configures clock, multiplex ratio, charge pump, addressing, orientation, contrast and COM pins.
OLED_Clear()	Writes 128 zero bytes to each of the eight 8-pixel pages.

The initialization sequence uses 0xAE to keep the display off during configuration, 0xA8/0x3F for 64 multiplexed rows, 0x8D/0x14 for the charge pump, 0x20/0x00 for addressing, 0xA1 and 0xC8 for orientation, 0xDA/0x12 for COM configuration, and 0xAF to enable the display. In the current character mapping, the letter “r” and the colon share one font index; unique glyphs require separate indices.

3.2 Sending commands with OLED_WriteCommand()

```
uint8_t data[2] = {0x00, cmd};
HAL_I2C_Master_Transmit(&hi2c1, OLED_ADDR, data, 2, 100);
```

The first byte is the SSD1306 control byte. A value of 0x00 tells the controller that the following byte is a command rather than display data. The second byte contains the actual command. The transaction is sent on I²C1 to OLED_ADDR with a 100 ms HAL timeout.

3.3 Sending display data with OLED_WriteData()

```
buffer[0] = 0x40;
memcpy(&buffer[1], data, size);
HAL_I2C_Master_Transmit(&hi2c1, OLED_ADDR, buffer, size + 1, 100);
```

The local buffer contains one control byte followed by a maximum of 128 display bytes. The value 0x40 marks the remaining bytes as graphics memory data. Limiting size to 128 prevents buffer overflow. Sending the control byte and payload in one I²C transaction reduces overhead.

3.4 Text rendering with OLED_Print()

OLED_Print() processes the string until the terminating null character is reached. Digits map directly to font indices 0–9. A switch statement selects glyphs for the required status letters, the colon and the space. Each glyph contains five column bytes for a 5 × 7 character. A sixth empty column is sent after every glyph to provide one pixel of spacing.

3.5 Clearing all display pages

```
for (uint8_t page = 0; page < 8; page++) {
    OLED_WriteCommand(0xB0 + page);
    OLED_WriteCommand(0x00);
    OLED_WriteCommand(0x10);
    OLED_WriteData(zero, 128);
}
```

The 64-pixel display height is divided into eight pages of eight pixels. For each page, the page command and low/high column commands return the write position to the left edge. Writing 128 zero bytes switches off every pixel in that page. Repeating the operation eight times clears the complete screen.

4. Slave Firmware and Phase-Control Timing

4.1 Dim value to firing delay

```
uint32_t Dim_To_Delay_us(uint8_t dim)
{
    if (dim == 0) return 11000;    // outside the half-cycle: do not fire
    if (dim > 95) dim = 95;
    if (dim >= 95) return 200;     // almost full output
    return 9800 - (((uint32_t)(dim - 1) * 9600) / 94);
}
```

A dim value of 0 intentionally produces a delay outside the usable 10 ms half-cycle. Values from 1 to 95 map approximately from 9.8 ms to 0.2 ms. A shorter delay transfers more of the half-wave to the load.

4.2 I²C receive callback

The slave receives two bytes. During address assignment, the first byte is compared with MY_ID and the second byte becomes the new address when ENABLE is active. During normal operation, 0x01 requests a five-byte status response, 0x02 updates dim_value1 and 0x03 updates dim_value2. Values above 95 are limited, a status LED is toggled, and interrupt reception is restarted.

```
void HAL_I2C_SlaveRxCpltCallback(I2C_HandleTypeDef *hi2c)
{
    if (hi2c->Instance == I2C1) {
        uint8_t cmd = rxData[0];
        uint8_t value = rxData[1];

        if (t == 0) {
            if (HAL_GPIO_ReadPin(enable_GPIO_Port, enable_Pin) == GPIO_PIN_SET &&
                cmd == MY_ID) {
                t = 1;
                Set_New_Address(value);
                return;
            }
            Restart_I2C_Receive();
            return;
        }

        if (cmd == 0x01) {
            pending_tx = 1;
            HAL_I2C_Slave_Transmit_IT(&hi2c1, txData, 5);
            return;
        } else if (cmd == 0x02) {
            if (value > 95) value = 95;
            dim_value1 = value;
        }
    }
}
```

```

        HAL_GPIO_TogglePin(led_GPIO_Port, led_Pin);
    } else if (cmd == 0x03) {
        if (value > 95) value = 95;
        dim_value2 = value;
        HAL_GPIO_TogglePin(led_GPIO_Port, led_Pin);
    }
    Restart_I2C_Receive();
}
}

```

4.3 Zero-cross and TIM2 callbacks

At each rising edge from the zero-cross detector, the firmware calculates the firing time for both channels, clears the fired flags, forces both gate outputs low, resets TIM2 and starts its interrupt. Each timer interrupt advances `tim2_us` by `TIMER_STEP_US`. When a channel reaches its firing time, its gate output is driven high for `TRIAC_PULSE_US` and then returned low. The timer stops once both pulses are complete or at the 9.9 ms safety limit.

```

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_15) {
        zero_count++;
        fire_time1 = Dim_To_Delay_us(dim_value1);
        fire_time2 = Dim_To_Delay_us(dim_value2);
        tim2_us = 0;
        triac1_fired = triac2_fired = 0;
        triac1_off_time = triac2_off_time = 0;
        HAL_GPIO_WritePin(triac1_GPIO_Port, triac1_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(triac2_GPIO_Port, triac2_Pin, GPIO_PIN_RESET);
        __HAL_TIM_SET_COUNTER(&htim2, 0);
        HAL_TIM_Base_Start_IT(&htim2);
    }
}

```

4.4 Timer interrupt and TRIAC gate pulses

```

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        tim2_us += TIMER_STEP_US;

        if (dim_value1 > 0 && !triac1_fired && tim2_us >= fire_time1) {
            HAL_GPIO_WritePin(triac1_GPIO_Port, triac1_Pin, GPIO_PIN_SET);
            triac1_fired = 1;
            triac1_off_time = tim2_us + TRIAC_PULSE_US;
        }
        if (triac1_fired && triac1_off_time && tim2_us >= triac1_off_time) {
            HAL_GPIO_WritePin(triac1_GPIO_Port, triac1_Pin, GPIO_PIN_RESET);
            triac1_off_time = 0;
        }
        if (dim_value2 > 0 && !triac2_fired && tim2_us >= fire_time2) {
            HAL_GPIO_WritePin(triac2_GPIO_Port, triac2_Pin, GPIO_PIN_SET);
            triac2_fired = 1;
            triac2_off_time = tim2_us + TRIAC_PULSE_US;
        }
        if (triac2_fired && triac2_off_time && tim2_us >= triac2_off_time) {
            HAL_GPIO_WritePin(triac2_GPIO_Port, triac2_Pin, GPIO_PIN_RESET);
            triac2_off_time = 0;
        }
        if (((triac1_fired || dim_value1 == 0) && triac1_off_time == 0 &&
            (triac2_fired || dim_value2 == 0) && triac2_off_time == 0) ||

```

```

    tim2_us >= 9900) {
        HAL_TIM_Base_Stop_IT(&htim2);
        HAL_GPIO_WritePin(triac1_GPIO_Port, triac1_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(triac2_GPIO_Port, triac2_Pin, GPIO_PIN_RESET);
    }
}

```

4.5 Combined slave-software sequence

The four software blocks operate in sequence. First, the slave receives a desired dim value over I²C and stores it for channel 1 or channel 2. At the next zero crossing, that value is converted into a delay. TIM2 then counts from zero. When the delay expires, the correct optotriac receives a short pulse. The next zero crossing repeats the process. Because each channel has its own dim value, firing time, fired flag and pulse end time, both channels can be controlled independently.

Timing principle At 50 Hz, a full period is 20 ms and a half-cycle is 10 ms. The TRIAC gate pulse is only a trigger; after latching, the device continues to conduct until load current falls below its holding current.

5. Zero-Cross Detection

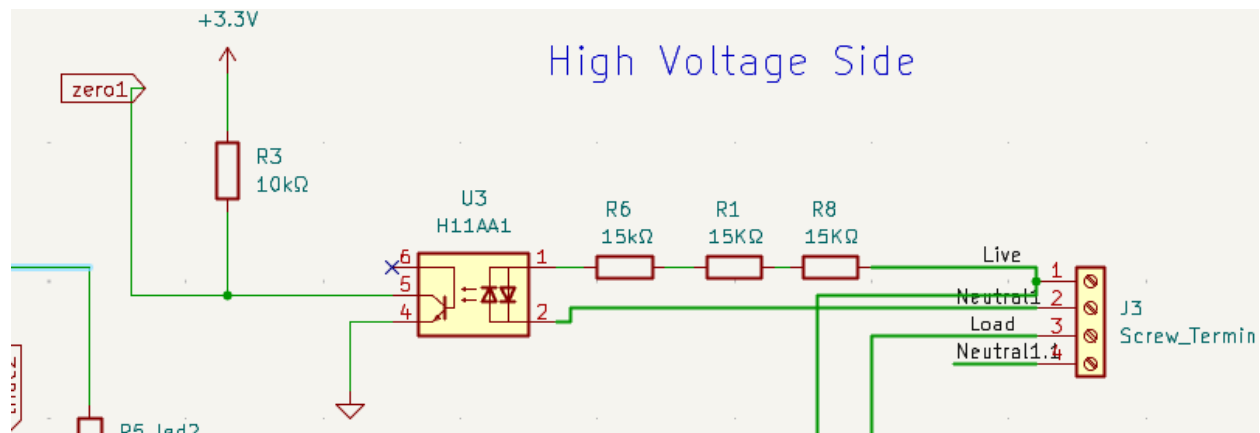


Figure 1 — H11AA1 zero-cross detector with pull-up and three series resistors.

The H11AA1 contains two antiparallel infrared LEDs and a phototransistor, so it can respond to both AC half-cycles. Away from the zero crossing, LED current turns the phototransistor on and pulls zero1 low. Near 0 V, LED current falls, the transistor turns off and the 10 kΩ pull-up raises zero1 to 3.3 V. A rising-edge external interrupt detects this transition.

Away from 0 V: H11AA1 LED ON -> transistor ON -> zero1 LOW
 Near 0 V: H11AA1 LED OFF -> transistor OFF -> pull-up -> zero1 HIGH

Calculation	Result
Total input resistance	$15\text{ k}\Omega + 15\text{ k}\Omega + 15\text{ k}\Omega = 45\text{ k}\Omega$
Approximate RMS current	$230\text{ V} / 45\text{ k}\Omega = 5.11\text{ mA}$

Calculation	Result
Mains peak voltage	$230 \times \sqrt{2} \approx 325 \text{ V}$
Approximate peak current	$(325 - 1.2) / 45 \text{ k}\Omega \approx 7.2 \text{ mA}$
Total resistor dissipation	$230^2 / 45 \text{ k}\Omega \approx 1.18 \text{ W}$
Dissipation per resistor	$\approx 0.39 \text{ W}$

Current through the H11AA1 input resistors

Step 1: Add the three series resistors: $15 \text{ k}\Omega + 15 \text{ k}\Omega + 15 \text{ k}\Omega = 45 \text{ k}\Omega$.

Step 2: Use Ohm's law: current = voltage $\div$ resistance.

Step 3: $I = 230 \text{ V} \div 45,000 \Omega = 0.00511 \text{ A}$.

Step 4: Convert amperes to milliamperes: $0.00511 \text{ A} \times 1000 = 5.11 \text{ mA}$.

Answer: Approximately 5.11 mA RMS flows through the resistor chain.

Power in each 15 k Ω resistor

Step 1: First calculate the total power: $P = V^2 \div R$.

Step 2: $P_{\text{total}} = 230^2 \div 45,000 = 1.18 \text{ W}$.

Step 3: The three equal resistors share the power: $1.18 \text{ W} \div 3 = 0.39 \text{ W}$.

Answer: Each resistor dissipates approximately 0.39 W, so a 0.25 W resistor is too small.

Design check A calculated 0.39 W per 15 k Ω resistor exceeds a typical 0.25 W rating. Verify both power and working-voltage ratings. The digital edge is close to, but not exactly at, 0 V because it depends on LED current, CTR, pull-up current and transistor characteristics.

6. TRIAC Power Stage

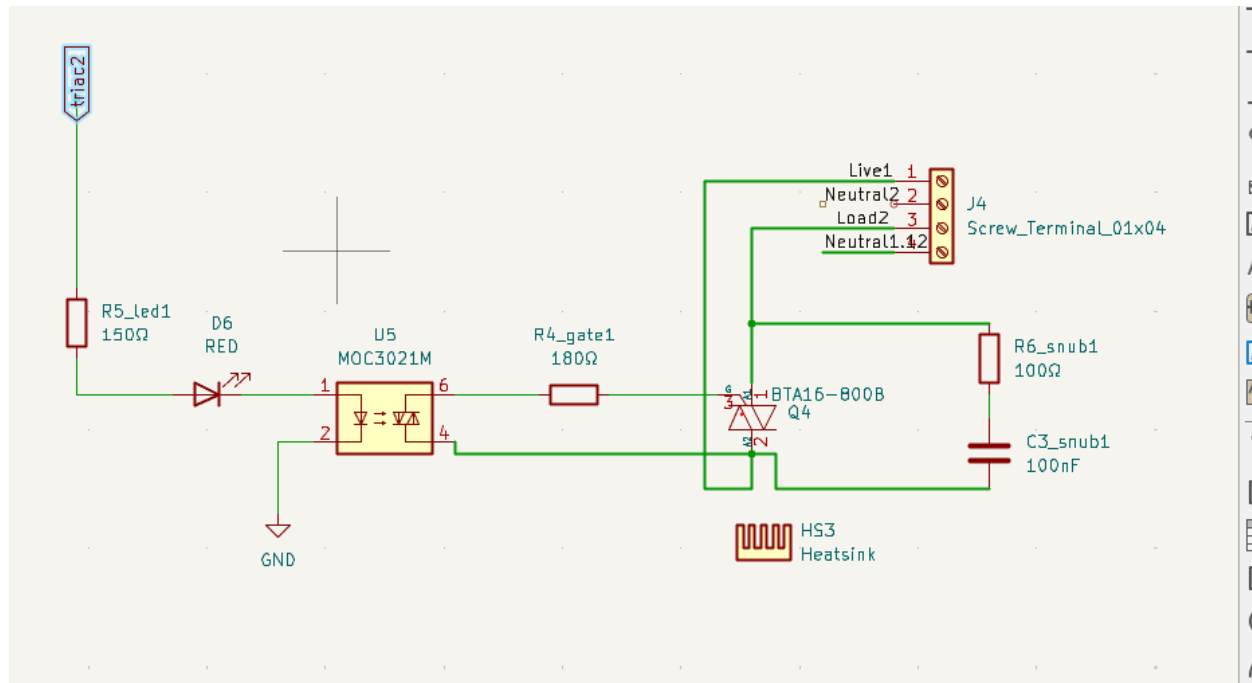


Figure 2 — Random-phase optotriac, BTA16 power TRIAC and RC snubber.

The STM32 drives the input LED of a random-phase optotriac. Its output provides galvanically isolated gate current to the BTA16-800B power TRIAC. A random-phase device is necessary because the microcontroller selects the firing angle; a zero-cross optotriac would prevent this control strategy.

6.1 Optotriac input

With D6 removed, R5 equal to 120 Ω, a 3.3 V GPIO output and an assumed optotriac LED forward voltage of 1.5 V:

Quantity	Calculation
Resistor voltage	$3.3 - 1.5 = 1.8 \text{ V}$
LED current	$1.8 / 120 = 15 \text{ mA}$
R5 dissipation	$0.015^2 \times 120 \approx 0.027 \text{ W}$
LED power	$1.5 \times 0.015 \approx 0.0225 \text{ W}$

MOC3021 LED current

Step 1: The STM32 output is 3.3 V and the optotriac LED uses approximately 1.5 V.

Step 2: Voltage across R5 = 3.3 V – 1.5 V = 1.8 V.

Step 3: Current = voltage ÷ resistance = 1.8 V ÷ 120 Ω = 0.015 A.

Step 4: Convert to milliamperes: 0.015 A × 1000 = 15 mA.

Answer: The MOC3021 input LED current is approximately 15 mA.

Minimum resistor power rating for R5

Step 1: Use $P = I^2 \times R$.

Step 2: $P = 0.015^2 \times 120 = 0.027 \text{ W}$.

Step 3: Choose a standard rating comfortably above this value.

Answer: A 0.125 W or 0.25 W resistor has sufficient power margin, provided pulse and temperature conditions are acceptable.

Component selection Confirm that the exact STM32 pin and I/O bank can source the required current. A 400 V optotriac has limited transient margin above the approximately 325 V peak of a 230 V RMS supply; select a suitably rated random-phase device for the final 230/240 V design.

6.2 Phase-angle control

Delay after zero crossing	Firing angle	Approximate resistive-load power
0.2 ms	3.6°	Almost 100%
3 ms	54°	Approximately 85%
5 ms	90°	Approximately 50%
8 ms	144°	Approximately 5%
9.8 ms	176.4°	Almost 0%

For a 100 W resistive load at 230 V, the RMS current is approximately 0.435 A, the equivalent resistance is 529 Ω and the peak current is approximately 0.615 A. Using a simplified on-state model gives an estimated TRIAC loss of about 0.34 W at 100 W and about 3.8 W at 1000 W. Final heatsink and copper-area requirements must be based on worst-case datasheet and thermal measurements.

Current of a 100 W lamp

Step 1: Use the power formula $P = V \times I$.

Step 2: Rearrange it: $I = P \div V$.

Step 3: $I = 100 \text{ W} \div 230 \text{ V} = 0.435 \text{ A}$.

Answer: A 100 W resistive lamp draws approximately 0.435 A RMS.

Equivalent resistance of a 100 W lamp

Step 1: Use $R = V^2 \div P$.

Step 2: $R = 230^2 \div 100$.

Step 3: $230^2 = 52,900$; $52,900 \div 100 = 529 \Omega$.

Answer: The hot operating resistance is approximately 529 Ω .

6.3 RC snubber

The 100 Ω / 100 nF snubber is connected across the power TRIAC. The capacitor temporarily carries current during fast voltage changes, reducing dV/dt , while the resistor limits and damps charge and discharge current. Its time constant is 10 μ s. At 50 Hz, the capacitor reactance is about 31.8 k Ω and the current is about 7.2 mA; this low-frequency value does not replace pulse and mains-safety rating checks.

RC snubber time constant

Step 1: Convert 100 nF to farads: 100 nF = 0.0000001 F.

Step 2: Use $\tau = R \times C$.

Step 3: $\tau = 100 \Omega \times 0.0000001 \text{ F} = 0.00001 \text{ s}$.

Step 4: Convert seconds to microseconds: 0.00001 s = 10 μ s.

Answer: The snubber time constant is 10 μ s.

50 Hz current through the 100 nF snubber capacitor

Step 1: Calculate capacitive reactance: $X_C = 1 \div (2 \times \pi \times f \times C)$.

Step 2: $X_C = 1 \div (2 \times 3.1416 \times 50 \times 100 \text{ nF}) \approx 31,800 \Omega$.

Step 3: Current = $230 \text{ V} \div 31,800 \Omega = 0.00723 \text{ A}$.

Step 4: Convert to milliamperes: 0.00723 A = 7.23 mA.

Answer: The approximate 50 Hz capacitor current is 7.2 mA.

7. Power-Supply and Protection Circuits

7.1 Reverse-polarity protection

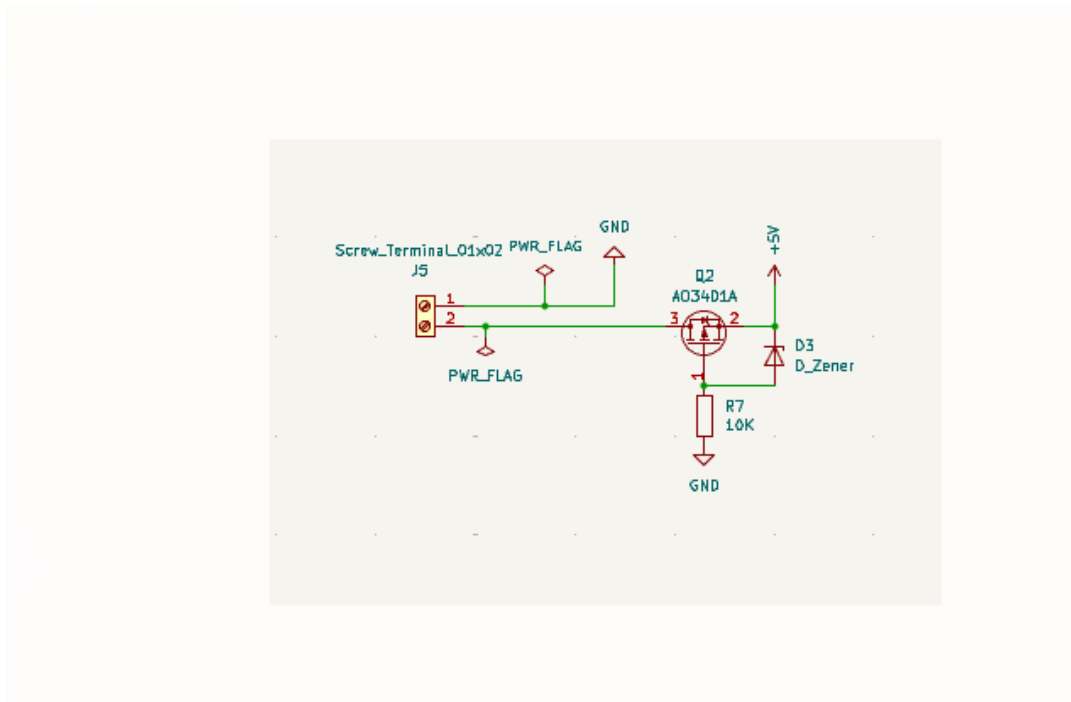


Figure 3 — AO3401A P-channel MOSFET reverse-polarity protection.

With correct polarity, R7 pulls the gate to ground while the source is near 5 V, producing $V_{GS} \approx -5$ V and turning the P-channel MOSFET on. After start-up, current flows mainly through the low-resistance MOSFET channel. The zener diode limits gate-source transients and must keep $|V_{GS}|$ below the device absolute maximum.

Load current	Drop at 60 mΩ	MOSFET loss
0.5 A	0.030 V	0.015 W
1.0 A	0.060 V	0.060 W
1.5 A	0.090 V	0.135 W
2.0 A	0.120 V	0.240 W

AO3401A voltage drop at 1 A

Step 1: Use $V_{drop} = I \times R_{DS(on)}$.

Step 2: $R_{DS(on)} = 60 \text{ m}\Omega = 0.060 \text{ }\Omega$.

Step 3: $V_{drop} = 1 \text{ A} \times 0.060 \text{ }\Omega = 0.060 \text{ V}$.

Answer: The MOSFET drops approximately 0.06 V at 1 A.

AO3401A power loss at 1 A

Step 1: Use $P = I^2 \times R_{DS(on)}$.

Step 2: $P = 1^2 \times 0.060 = 0.060 \text{ W}$.

Answer: The MOSFET dissipates approximately 0.06 W at 1 A.

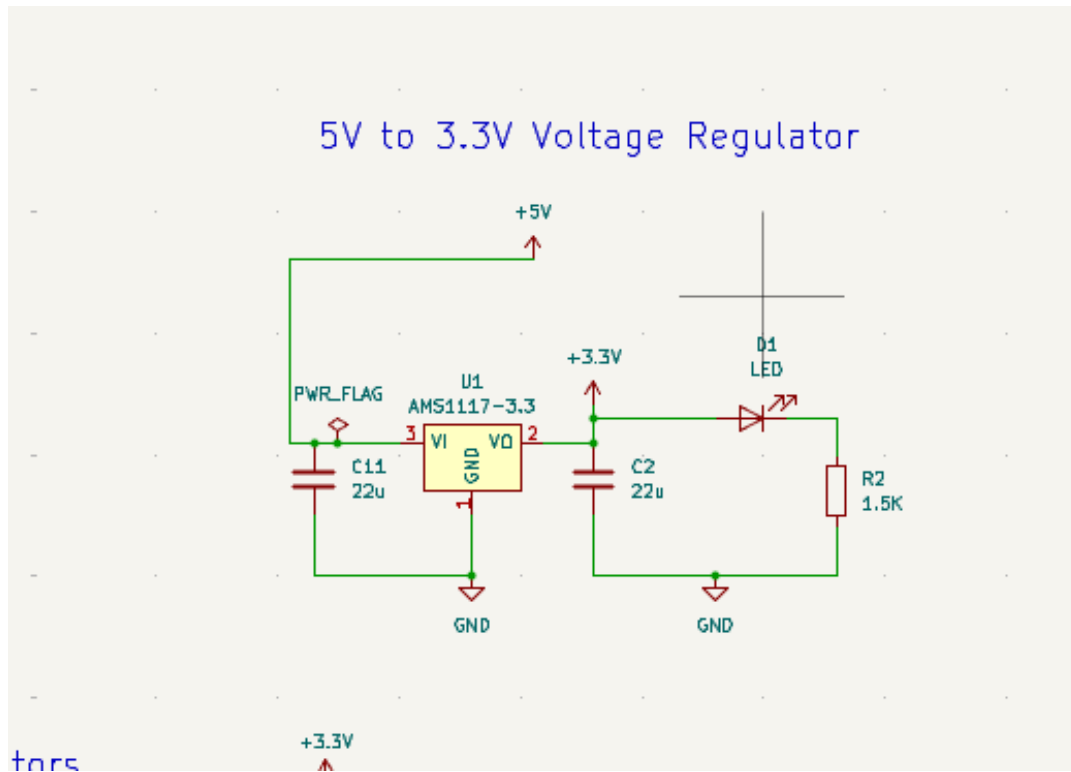
7.2 5 V to 3.3 V linear regulator

Figure 4 — AMS1117-3.3 regulator, decoupling capacitors and status LED.

The AMS1117-3.3 dissipates the 1.7 V difference between 5 V and 3.3 V as heat. Approximate loss is $(V_{in} - V_{out}) \times I$. The ideal voltage efficiency is about 66%. C1 and C2 support stability and transient response. With a red LED of approximately 2.0 V and $R2 = 1.5 \text{ k}\Omega$, indicator current is about 0.87 mA.

3.3 V load current	Approximate regulator loss
0.10 A	0.17 W
0.30 A	0.51 W
0.50 A	0.85 W
1.00 A	1.70 W

AMS1117 heat loss at 300 mA

Step 1: Voltage removed by the regulator = $5.0\text{ V} - 3.3\text{ V} = 1.7\text{ V}$.

Step 2: Convert 300 mA to amperes: $300\text{ mA} = 0.30\text{ A}$.

Step 3: Use $P_{\text{loss}} = \text{voltage difference} \times \text{current}$.

Step 4: $P_{\text{loss}} = 1.7\text{ V} \times 0.30\text{ A} = 0.51\text{ W}$.

Answer: The AMS1117 converts approximately 0.51 W into heat at a 300 mA load.

Current through the 3.3 V status LED

Step 1: Assume a red LED voltage of 2.0 V.

Step 2: Voltage across the 1.5 k Ω resistor = $3.3\text{ V} - 2.0\text{ V} = 1.3\text{ V}$.

Step 3: $I = 1.3\text{ V} \div 1500\text{ }\Omega = 0.000867\text{ A}$.

Step 4: Convert to milliamperes: $0.000867\text{ A} = 0.87\text{ mA}$.

Answer: The indicator LED current is approximately 0.87 mA.

7.3 12 V to 5 V synchronous buck converter

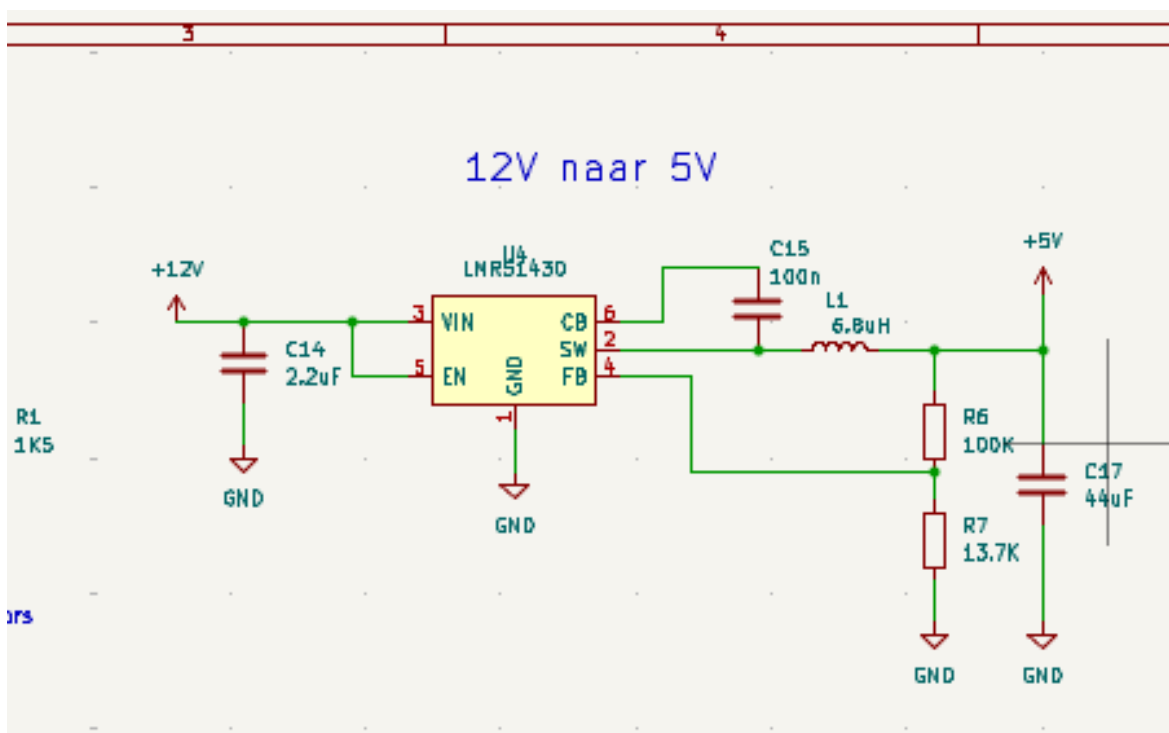


Figure 5 — LMR51430 synchronous buck converter configured for approximately 5 V.

The LMR51430 converts the 12 V input efficiently to 5 V. During the high-side MOSFET on-time, inductor current rises and energy is stored in L1. During the off-time, the synchronous low-side MOSFET provides a current path and the inductor continues supplying the load. The output capacitors smooth the voltage and provide local transient current.

Item	Role or result
------	----------------

Item	Role or result
C14 = 2.2 μ F	Local input bypass for the high di/dt loop.
C15 = 100 nF	Bootstrap capacitor for the high-side gate driver.
L1 = 6.8 μ H	Stores magnetic energy and smooths current.
C17 = 44 μ F	Filters output ripple and supplies load transients.
R6 = 100 k Ω / R7 = 13.7 k Ω	Feedback divider: $V_{out} \approx 0.6 \times (1 + 100/13.7) \approx 4.98$ V.
Ideal duty cycle	$5 / 12 \approx 41.7\%$; at 500 kHz, ideal on-time ≈ 0.83 μ s.
Estimated inductor ripple	Approximately 0.86 A peak-to-peak.
Ideal capacitive ripple	Approximately 4.9 mV, excluding ESR, ESL and control-loop effects.

Ideal buck-converter duty cycle

Step 1: Use $D = V_{out} \div V_{in}$.

Step 2: $D = 5$ V $\div$ 12 V = 0.417.

Step 3: Convert to percent: $0.417 \times 100 = 41.7\%$.

Answer: The ideal high-side MOSFET duty cycle is approximately 41.7%.

Buck-converter switching period and on-time

Step 1: At 500 kHz, the period is $T = 1 \div 500,000 = 0.000002$ s = 2.0 μ s.

Step 2: On-time = duty cycle $\times$ period.

Step 3: $t_{on} = 0.417 \times 2.0$ μ s = 0.834 μ s.

Answer: The ideal on-time is approximately 0.83 μ s per switching cycle.

Buck feedback voltage

Step 1: Use $V_{out} = V_{ref} \times (1 + R_{top} \div R_{bottom})$.

Step 2: $V_{out} = 0.6 \times (1 + 100$ k $\Omega \div 13.7$ k Ω).

Step 3: $100 \div 13.7 = 7.30$; therefore $V_{out} = 0.6 \times 8.30$.

Step 4: $0.6 \times 8.30 = 4.98$ V.

Answer: The selected feedback resistors set the output to approximately 4.98 V.

8. PCB Implementation

The hardware is divided into a low-voltage master PCB and a local slave/dimmer PCB. This division preserves modularity and helps separate central logic from the mains power electronics.

8.1 Master PCB

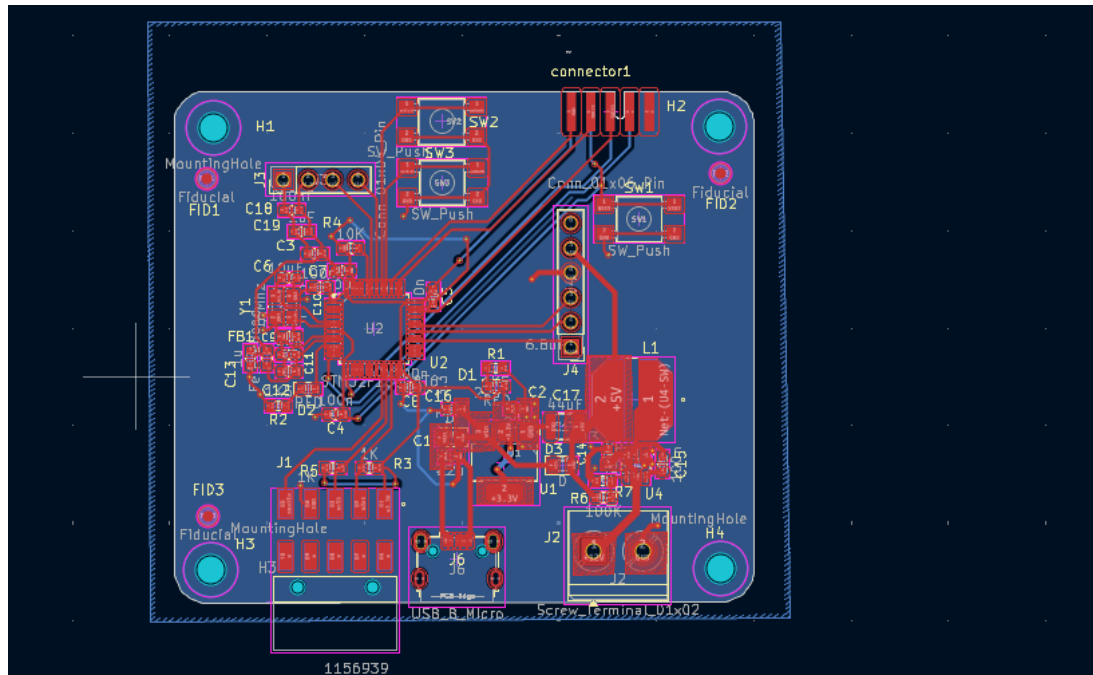


Figure 6 — Master PCB layout.

The master PCB places the STM32 centrally, with short routes to the display, communication headers, push buttons, Bluetooth interface and programming connector. Power conversion and distribution are grouped around the board edges. The USB and screw-terminal interfaces remain accessible for development and installation.

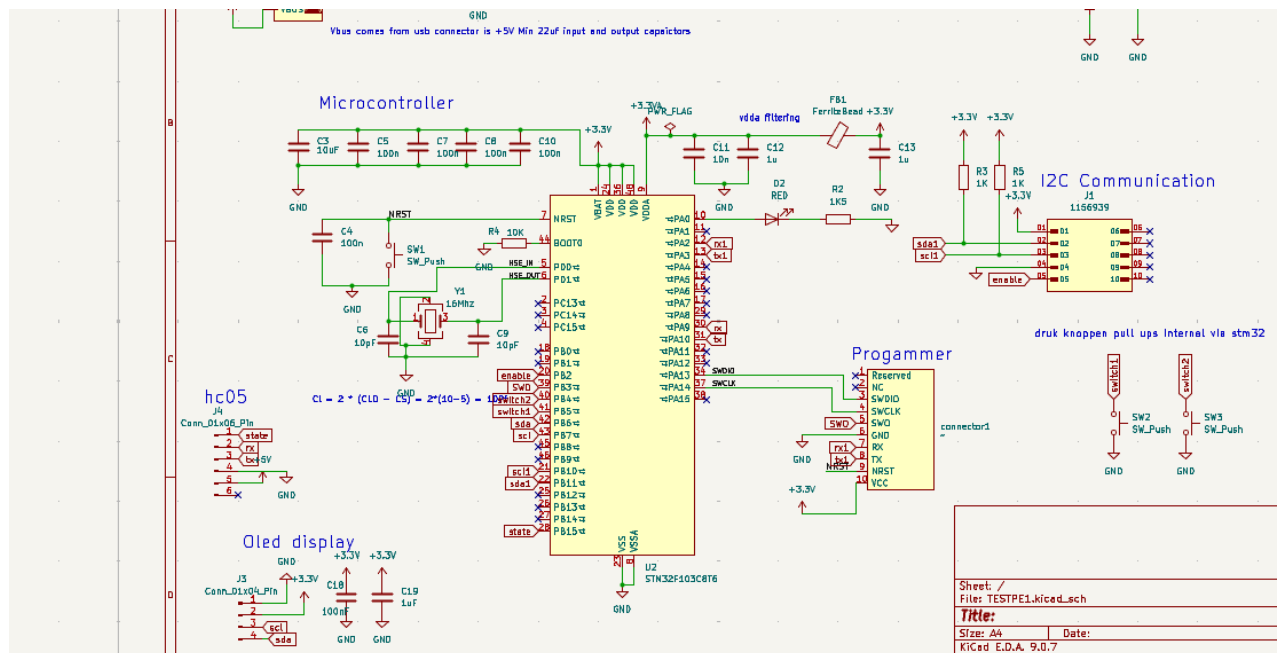


Figure 7 — Master-node schematic and peripheral connections.

8.2 Slave/dimmer PCB

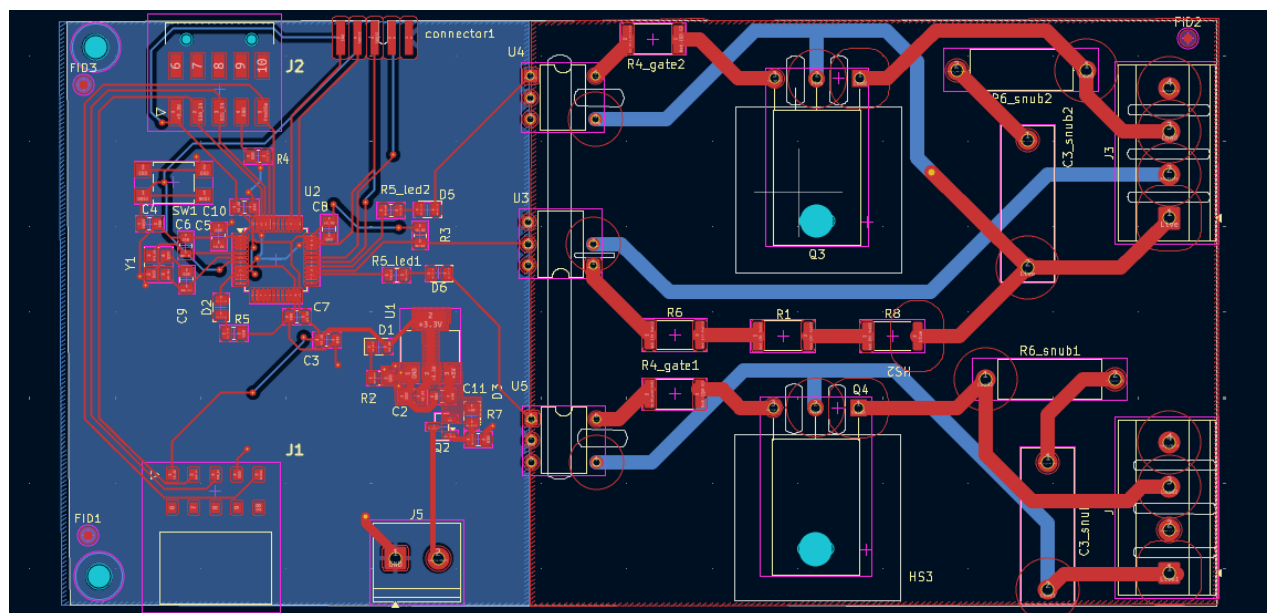


Figure 8 — Slave/dimmer PCB with low-voltage control and two mains channels.

The slave board contains the MCU-side I²C interface, zero-cross detector and two optically isolated TRIAC channels. The low-voltage logic and mains copper are physically separated. A routed slot increases the surface path between voltage domains. High-current traces, terminal blocks, TRIAC footprints and heatsink clearances are placed on the power side.

PCB safety For the current prototype, a minimum creepage/clearance target of 2.5 mm was considered. This value is not universally sufficient: required distances depend on working voltage, pollution degree, material group, overvoltage category, altitude, coating and the applicable standard. The final design must be checked against IEC 60664-1 and the intended product standard.

8.3 Detailed master-board hardware

8.3.1 STM32 microcontroller and decoupling

The STM32 is the central controller of the master board. It processes local buttons and Bluetooth commands, maintains the OLED user interface and controls the slave bus. The 3.3 V supply is connected to all required VDD pins and the analogue supply pin. Local decoupling capacitors are placed close to the power pins to supply short current peaks and prevent digital switching noise from disturbing the MCU or communication buses.

8.3.2 Reset, BOOT0 and external clock

The reset input is held in its normal inactive state and can be pulled active with the reset push button. This permits a controlled restart without removing the power supply. BOOT0 is held low for ordinary execution from internal Flash. The external crystal and its load capacitors provide an accurate clock source when the firmware configuration selects it; short, symmetrical traces reduce noise and parasitic capacitance.

8.3.3 OLED interface on I²C1

The OLED connector carries 3.3 V, ground, SCL and SDA. PB6 and PB7 form the I²C1 clock and data lines. Because I²C uses open-drain outputs, the bus requires pull-up resistors. The display uses address 0x3C in the implemented configuration and receives both controller commands and graphics data through the same two-wire interface.

8.3.4 Slave communication on I²C2

PB10 and PB11 form the second I²C peripheral and are routed to the slave connectors. Keeping the OLED on I²C1 and the slave network on I²C2 separates user-interface traffic from real-time dimmer communication. The ENABLE line is routed separately so the master can place compatible slave boards into their address-assignment state.

8.3.5 Local push buttons

SW2 is connected to switch1 on PB5 and SW3 to switch2 on PB4. Each button connects its GPIO input to ground when pressed. Internal pull-ups therefore produce a stable logic high when released and a logic low when pressed. Firmware debouncing and edge detection prevent one physical press from being interpreted as multiple commands.

8.3.6 HC-05 Bluetooth module

The HC-05 connector includes supply, ground, serial RX/TX and the STATE signal. The UART link transfers commands from the mobile application and can return system status. STATE allows the MCU to distinguish a connected link from an idle module. In the intended control logic, local buttons can be disabled while Bluetooth is connected to avoid competing commands.

8.3.7 Programming and debugging interface

The programming connector exposes SWDIO on PA13, SWCLK on PA14, reset, power and ground. An ST-Link can program the MCU, stop execution at breakpoints, inspect registers and memory, and step through the firmware. This interface is essential for commissioning the communication, display and dimming functions independently.

8.3.8 Master connection summary

Interface	STM32 connection	Purpose
OLED I ² C1	PB6 / PB7	Display commands and graphics data
Slave I ² C2	PB10 / PB11	Addressing, polling and dim commands
ENABLE	PB2	Slave address-assignment control
Local buttons	PB5 / PB4	Manual dimmer control
Bluetooth UART	Serial RX / TX	Mobile-app commands and status
HC-05 STATE	GPIO input	Bluetooth connection detection
SWD	PA13 / PA14	Programming and debugging

9. Mechanical and Enclosure Design

Custom enclosures were designed around the PCB geometry and the safety requirements of a mixed low-voltage and 230 V AC system. Both housings use a common visual language, internal PCB standoffs and a two-piece closure concept.

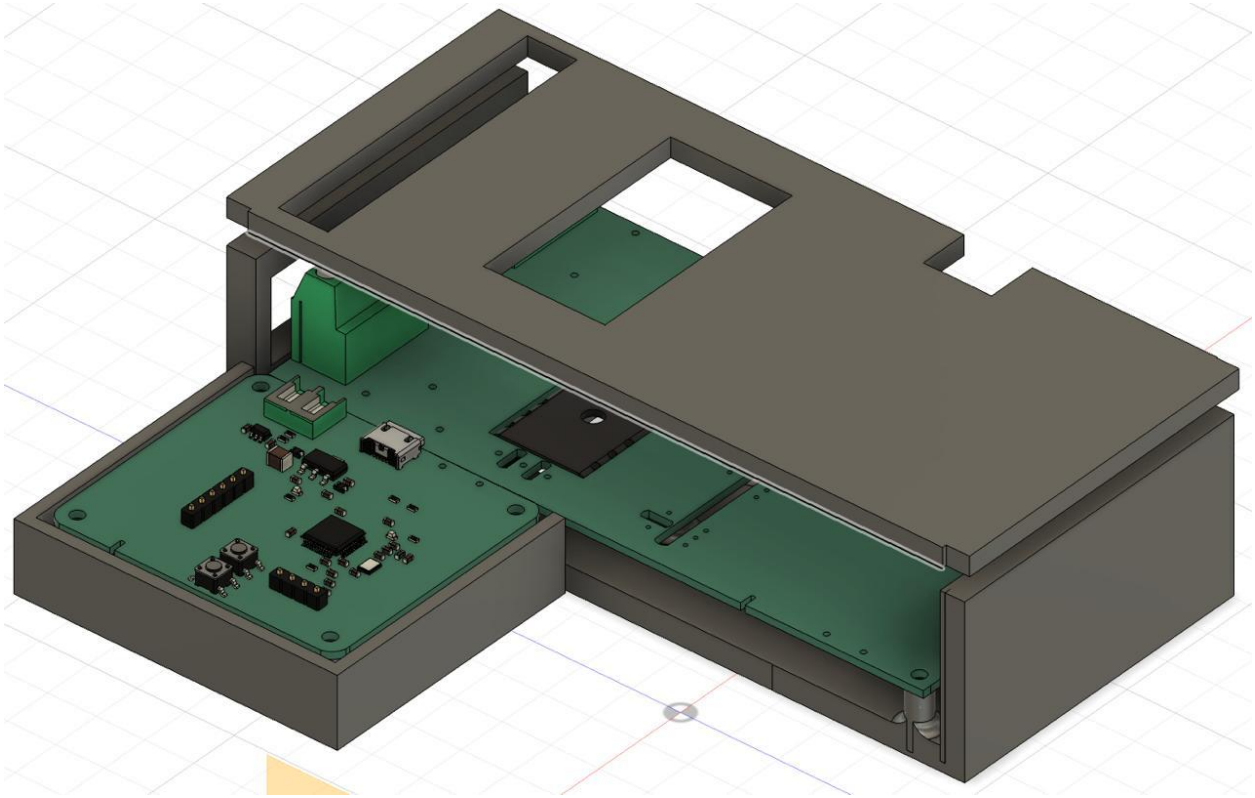


Figure 9 — 3D enclosure assembly showing the master and slave modules with installed PCBs.

9.1 Master-node enclosure

The master housing uses an asymmetrical L-shaped footprint tailored to the central control PCB. The top panel has a rectangular OLED cutout for a flush user interface. Side openings provide access to the Micro-USB connector and power terminals. The enclosure height provides clearance for vertical pin headers, the Bluetooth module, the I²C daisy-chain connectors and the tactile push buttons.

Mechanical parameter	Value
Mass	227.599 g
Volume	28,993.521 mm ³
Density	0.008 g/mm ³
Surface area	21,446.653 mm ²
Centre of mass (X, Y, Z)	-36.043 mm, 24.694 mm, -51.132 mm

9.2 Slave/dimmer enclosure

The slave housing uses a compact rectangular form so multiple units can be stacked or mounted in sequence. The architecture accommodates the PCB isolation slot, while ventilation openings above the power TRIACs support passive heatsink cooling. The high-voltage end remains accessible for thick-gauge mains conductors and terminal blocks without mechanical interference.

Mechanical parameter	Value
Mass	942.964 g
Volume	120,100.000 mm ³
Density	0.008 g/mm ³
Surface area	80,834.756 mm ²
Bounding box (L × W × H)	68.775 mm × 44.200 mm × 152.787 mm
Centre of mass (X, Y, Z)	26.601 mm, 37.801 mm, -10.669 mm

9.3 Assembly and mounting

- Cylindrical standoffs at the base corners raise the PCBs and prevent through-hole solder joints from touching the enclosure floor.
- A lip-and-groove or friction-fit track allows the top and base to slide or snap together without immediate dependence on threaded fasteners.
- Terminal openings must include strain relief and finger-safe protection in the final product.
- Ventilation geometry must not compromise touch protection, insulation spacing or flame containment.
- The enclosure material and wall thickness must be selected for temperature, impact, flammability and electrical-insulation requirements.

10. Development Problems and Troubleshooting

Several integration problems appeared while the project was being tested. These problems were important because each subsystem worked individually, but combining Bluetooth, two I²C slaves, oscilloscope measurements and mains-related hardware introduced interactions that were not visible during isolated tests.

10.1 Unstable communication with two I²C slaves

The master could initially communicate with one slave, but adding the second slave sometimes caused one node to disappear or caused the I²C bus to remain busy. In some tests a slave became available only after an additional reset, and after repeated switching cycles the second node could go offline. The objective was to connect both slaves without losing communication between the master and either slave.

Possible cause	Effect on the bus	Project response
Both slaves start with the	Two devices can respond	Use ENABLE, unique hardware IDs and

Possible cause	Effect on the bus	Project response
same address	simultaneously during start-up.	dynamic addresses 0x12 and 0x13.
Slave reset during a transfer	SDA may remain low and the HAL peripheral remains BUSY.	Generate nine SCL pulses, create a STOP condition and reinitialize I ² C2.
Address assignment too fast	The slave may not finish reinitializing before the next command.	Add controlled delays around ENABLE and address transmission.
Noise from AC switching	Fast TRIAC edges can disturb long low-voltage communication wiring.	Keep I ² C wiring short, use correct pull-ups, decoupling and physical separation.
Weak or missing common ground	Logic thresholds become unreliable.	Connect a solid low-voltage ground between master and slaves.

The firmware therefore assigns addresses only to missing nodes, polls both slaves periodically and marks a node unavailable when a transmit or receive operation fails. The I²C recovery function prevents one interrupted transaction from permanently blocking later communication. Hardware improvements such as shorter wiring, local decoupling and separation from TRIAC traces remain important because software recovery cannot remove electrical noise.

10.2 HC-05 TX measurement of approximately 1.5 V

The HC-05 appeared to transmit data because a logic analyzer decoded the UART characters, while a multimeter showed only approximately 1.5 V on the TX line. A fixed 1.5 V HIGH level would normally be too low for a 3.3 V STM32 input to be recognized reliably. However, a multimeter on an active UART signal usually displays an average voltage rather than the true HIGH level. A line alternating between 0 V and approximately 3.3 V can therefore produce an intermediate reading such as 1.5 V.

Instrument	What it showed	Correct interpretation
Logic analyzer	Recognizable UART bit pattern and characters.	Timing and data existed, but threshold settings could still hide a weak signal.
Multimeter	Approximately 1.5 V average.	Not enough to determine the UART HIGH amplitude while data is switching.
Oscilloscope	Instantaneous waveform, HIGH level, LOW level, noise and edge quality.	Best instrument for deciding whether TX actually reaches a valid logic HIGH.

The correct test is to measure HC-05 TX with an oscilloscope referenced to the same low-voltage ground while the module sends a repeating character pattern. The waveform should be inspected for its actual HIGH plateau, not only its average. If the HIGH level is truly around 1.5 V even when the line is idle, possible causes include an overloaded TX output, incorrect supply voltage, a damaged module, a short or low-resistance load, incorrect grounding or a damaged STM32 input pin.

Technical conclusion The 1.5 V multimeter reading alone does not prove that the HC-05 logic HIGH was 1.5 V. The logic analyzer confirmed activity, while the oscilloscope waveform is needed to verify the real voltage levels and noise margin.

10.3 STM32 damage during oscilloscope measurement

During testing, the master STM32 stopped operating after an oscilloscope ground connection was made. A standard bench oscilloscope ground clip is normally connected to protective earth. It is therefore not a floating reference. If the clip is attached to a point that is not at earth potential, or to circuitry with an unintended mains-related potential, the clip can create a high current path through the PCB, USB connection, programmer or MCU ground network.

The observed failure is consistent with damage caused by an incorrect ground reference, a ground loop, an overvoltage transient or current entering an STM32 pin beyond its supply rails. It cannot be proven from the failure alone that the oscilloscope ground was the only cause, but the timing of the event makes it the leading suspected cause.

Oscilloscope safety Never connect a grounded oscilloscope clip to an unknown point in a 230 V circuit. Verify the reference with the power removed, use galvanic isolation where appropriate, and use a correctly rated differential probe for non-earth-referenced or mains measurements. Do not defeat protective earth on the oscilloscope.

10.4 Lessons learned from integration

- Test the master, each slave and the HC-05 separately before combining them.
- Verify DC supply rails and ground continuity before connecting communication cables.
- Use the oscilloscope to measure real UART HIGH and LOW levels; do not interpret an active signal only from its multimeter average.
- Connect logic-analyzer and oscilloscope grounds only to the verified low-voltage ground domain.
- Keep I²C wiring short and away from TRIAC outputs and 230 V traces.
- Add one subsystem at a time and record which change introduces the fault.
- Use current-limited supplies and keep the ST-Link or USB connection isolated from unsafe mains test configurations.

11. Integrated System Behaviour

During operation, the master continuously maintains the user interface, communication and slave status. A valid Bluetooth or local command changes one selected dim value. The master transmits that value over I²C. On the following zero crossing, the slave calculates a new firing time and applies it independently to the requested channel. The 12 V input is converted to 5 V and then 3.3 V for the logic circuitry, while optical isolation separates the MCU gate command from the TRIAC power circuit.

Design checkpoint	Reason
Zero-cross input resistors	Verify dissipation and voltage rating of every series resistor.
Optotriac input	Verify LED current across GPIO voltage, temperature and component tolerance.
Optotriac blocking voltage	Provide adequate margin above mains peak and transient voltage.

Design checkpoint	Reason
Snubber components	Use mains- and pulse-rated components.
TRIAC thermal design	Calculate heatsink and copper requirements at maximum load.
MOSFET protection	Verify orientation, zener value and $ V_{GS} $ limit.
AMS1117 thermal design	Check junction temperature at maximum 3.3 V load.
Buck-converter layout	Keep the switch node, bootstrap loop, input capacitor and high di/dt loop compact.
Enclosures	Validate touch protection, strain relief, ventilation, mounting and flammability.

12. Verification and Recommended Tests

- Verify each power rail with a current-limited isolated supply before installing the mains section.
- Check I²C address assignment and status polling with one slave, then with multiple slaves.
- Capture the H11AA1 output and confirm one repeatable edge every 10 ms at 50 Hz.
- Measure gate-pulse delay and width at minimum, midpoint and maximum dim settings.
- Test first with a suitable resistive load and an isolation transformer; do not begin with an unknown LED driver.
- Monitor TRIAC, regulator, buck converter and enclosure temperatures at maximum intended load.
- Perform conducted and radiated EMC checks and confirm that the snubber does not create excessive leakage current.
- Carry out insulation, creepage, clearance, protective-earth and touch-access testing appropriate to the final construction.

13. Conclusion

The project demonstrates a complete modular lighting-control chain: Bluetooth and local input, embedded parsing and display, I²C distribution, dynamic slave addressing, synchronized phase-angle control, power conversion, PCB integration and custom mechanical protection. The architecture is scalable and technically coherent. Before use as a finished 230 V product, the prototype requires final component-margin checks, thermal validation, enclosure-safety verification and standards-based electrical testing.

14. References

- [1] Vishay — H11AA1 AC-input phototransistor optocoupler datasheet.
- [2] onsemi — MOC3010M/MOC3020M random-phase TRIAC-driver datasheet.
- [3] STMicroelectronics — BTA16 16 A TRIAC datasheet.
- [4] Alpha & Omega Semiconductor — AO3401A P-channel MOSFET datasheet.

[5] Texas Instruments — LMR51430 synchronous buck-converter datasheet and layout guidance.

[6] Advanced Monolithic Systems — AMS1117 regulator datasheet.

[7] IEC 60664-1 — Insulation coordination for equipment within low-voltage supply systems.

[8] Texas Instruments — Guidance on clearance and creepage for high-voltage applications.

Calculation note: where a component value was not fully specified in the source schematic, the assumption is stated. Final selection must use worst-case datasheet values, temperature limits, tolerance analysis and the manufactured PCB layout.